A Validation-Based Framework for Reasoning Reliability in Large Language Models

Without Model Retraining

Bingjie Wu

Master's Degree Project in
Computer and Systems Sciences
Spring term 2026
Supervisor: Hercules Dalianis
Department of Computer
and Systems Sciences



Stockholm
University

Abstract

Introduction Large Language Models (LLMs) generate fluent responses but often produce reasoning errors due to unsupported inference. Existing approaches mainly improve answer correctness but do not ensure that reasoning is supported by available information. This study focuses on improving reasoning reliability through explicit validation.

Research Question This study investigates: "Can the reasoning reliability of LLMs be improved, without retraining the model, by introducing external validation mechanisms that regulate the reasoning process?" and "How can such mechanisms be designed and integrated into the reasoning process to ensure that generated reasoning is supported by available information?"

Method A quantitative experimental design evaluates a validation-based reasoning control framework. The framework consists of structured representation, reasoning generation, and validation-based control. Input and reasoning are represented as graphs, and unsupported reasoning triggers revision or refusal. Experiments use the NeuLR and CLUTRR datasets with GPT-5, evaluated by correctness, error, and refusal rates.

Results The framework reduces the answer error rate across tasks. Correctness improves for inductive and abductive reasoning but decreases for deductive reasoning and CLUTRR. The refusal rate increases, indicating that unsupported answers are filtered.

Discussion External validation improves reasoning reliability by reducing unsupported answers without retraining. However, it does not consistently improve accuracy and is more effective for structurally regular tasks. Performance is limited for tasks requiring implicit reasoning, and the multi-stage pipeline introduces significant computational cost.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor Hercules Dalianis, for his patient guidance, detailed feedback, and strong sense of responsibility throughout this thesis project. His support, encouragement, and thoughtful suggestions played an essential role in helping me complete this research.

Before taking Professor Dalianis's Natural Language Processing course, I had never seriously considered working on an AI-related project. The course changed my perspective and made me realize that the field was far more approachable and intellectually engaging than I had previously imagined. Through this experience, I gradually developed the initial idea for this thesis.

When I first discussed this thesis idea with him, I only intended to ask whether such a project would even be feasible. I did not expect that he would be willing to supervise the project and guide me throughout the entire research process. During the development of the thesis, many aspects of the framework became significantly more complex than I had originally expected. Several parts of the design had to be repeatedly revised during implementation and experimentation, and there were many moments when I felt overwhelmed and even considered giving up. However, with his continuous guidance and support, I was able to gradually clarify the research direction and improve the work step by step. Without his help, this thesis would have been very difficult to complete.

I would also like to thank my classmates Sachinthani, Udeshi, Anton, and Christofher for their valuable feedback and discussions during the seminar sessions and thesis revision process. In particular, Sachinthani and Udeshi provided very careful suggestions regarding formatting and academic writing structure, which helped improve the overall presentation of the thesis.

I am especially grateful to Junqi Hu for many insightful discussions throughout this project. His encouragement, optimism, and technical discussions

provided significant inspiration during the development of this research. In particular, early discussions with him on reasoning verification and detection mechanisms helped shape the initial idea of the validation design used in this thesis.

In addition, I would like to thank Professor Aron Henriksson for reviewing the thesis and providing detailed and highly valuable suggestions, which helped me further improve and clarify the structure of the thesis.

I am also grateful to my friend Weiyi Yang for the conversations and support throughout my studies, which encouraged me to reflect more critically on both my work and myself.

Finally, I would like to express my deepest gratitude to my family.

I am especially grateful to my mother, Ms. Shuzhen Tang, for her unwavering encouragement and support throughout my decision to study abroad. When I was uncertain, anxious, and repeatedly doubted whether I could adapt to studying and living in a fully English-speaking environment, she continued to encourage me to move forward and never asked me to give up. Without her support and belief in me, I would not have had the courage to continue this journey.

I would also like to sincerely thank my sister, Ms. Xu Tang, for her direct and substantial support throughout my studies. She provided significant financial support during my education in Sweden and at many stages of my life. Without the support of my mother and sister, this thesis would not have been possible.

I am also grateful to all the friends, online communities, and people who encouraged and supported me along the way. In particular, I would like to thank my longtime friend Hongqian Li. Despite the long distance between us, her friendship and support remained an important source of comfort during my years in Sweden.

I would also like to thank Sweden itself.

Although my time here included many difficulties and moments of frustration, it also changed me in ways I never expected. During these years, I gradually rebuilt my confidence in communicating, presenting, networking, and expressing complex ideas in English. Looking back, this experience has become one of the most important periods of growth in my life.

Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
LLM(s)	Large Language Model(s)
CoT	Chain-of-Thought
KG(s)	Knowledge Graph(s)
GCR	Graph-constrained Reasoning
GoT	Graph of Thoughts
NLU	Natural Language Understanding

List of Figures

3.1 Example of a Preprocessed NeuLR Deductive Reasoning Sample 16
3.2 Example of a Preprocessed NeuLR Inductive Reasoning Sample 16
3.3 Example of a Preprocessed NeuLR Abductive Reasoning Sample 17
3.4 Example of a Preprocessed CLUTRR Sample (clutrr_mixed) . 18
3.5 Overall pipeline of the proposed validation-based reasoning control framework. 21

4.1 Comparison of Correctness Rates Between Baseline and Framework Settings 35
4.2 Comparison of Error Rates Between Baseline and Framework Settings 36
4.3 Comparison of Refusal Rates Between Baseline and Framework Settings 36

List of Tables

4.1 Overall Results for Baseline and Framework Settings 34

Contents

Abstract **iii**

Acknowledgements **v**

Abbreviations **vii**

List of Figures **ix**

List of Tables **xi**

Contents **xiii**

1 Introduction **1**
1.1 Background 1
1.2 Research Question 2

2 Extended Background **5**
2.1 Limitations of Reasoning Capabilities of LLMs 5
2.2 Existing Approaches to Improving Reasoning Capabilities of LLMs 6
2.3 Negative Effects of Improving Reasoning Capabilities of LLMs 7
2.4 Graph-based Approaches for Reasoning and Reasoning Verification 8
2.5 Reasoning Tasks and Datasets 9

3 Method **11**
3.1 Research Strategy 11
3.2 Research Method 12
3.2.1 Structured Representation 12
3.2.2 Validation-Based Reasoning Control 13
3.2.3 Reasoning Revision and Refusal Mechanism 13
3.2.4 Relation to Existing Approaches 14

3.2.5 Model Independence . . . 15
3.3 Data Collection Method . . . 15
3.4 Base Model . . . 18
3.5 Data Analysis Method . . . 19
3.6 Framework Implementation . . . 20
3.6.1 Information and Reasoning Representation . . . 22
3.6.2 Two-Round Reasoning Pipeline . . . 24
3.6.3 Task-Adaptive Validation Mechanisms . . . 25
3.6.4 External Relation Expansion . . . 29
3.6.5 Question-Focused Graph . . . 30
3.6.6 Refusal Detection . . . 30
3.7 Experimental Setup . . . 31
3.8 Ethical Considerations . . . 31

4 Results **33**
4.1 Baseline Experiment Results . . . 33
4.2 Framework Experiment Results . . . 33
4.3 Comparative Results . . . 34
4.4 Summary of Results . . . 34

5 Analysis **37**
5.1 Overall Effect of the Framework . . . 37
5.2 Task-wise Analysis . . . 37
5.2.1 Deductive Reasoning . . . 37
5.2.2 Inductive Reasoning . . . 38
5.2.3 Abductive Reasoning . . . 39
5.2.4 CLUTRR . . . 39
5.2.5 Summary of Task-wise Analysis . . . 40

6 Discussion **41**
6.1 Comparison with Previous Research . . . 41
6.2 Overall Discussion . . . 42
6.3 Computational Cost . . . 43
6.4 Societal Consequences . . . 44
6.5 Future Work . . . 44
6.6 Use of AI Tools . . . 44

Bibliography **48**

1

Introduction

1.1 Background

In recent years, with the rapid development of Artificial Intelligence (AI), a variety of models have emerged, among which Large Language Models (LLMs) are widely popular. After users input text into LLMs, the models can generate corresponding responses. However, these responses are not always correct; sometimes they may be inconsistent with objective facts or contain unreasonable content. Many researchers refer to this phenomenon as "AI hallucination" (Zhang et al. 2025), while some researchers question the appropriateness of this term (Maleki et al. 2024). In this study, a broad definition of hallucination is not adopted. Instead, this study focuses on a more specific category of errors, referred to as reasoning errors, which occur when the model produces incorrect answers due to failures in the reasoning process, such as skipping necessary steps, misusing given information, or introducing unsupported inferences.

Improving the logical reasoning capability of LLMs is an effective way to reduce reasoning errors. Existing studies mainly use three approaches, namely fine-tuning, prompting, and solver-based methods, to improve the logical reasoning capability of LLMs (Cheng et al. 2025). However, fine-tuning requires retraining the model and involves additional training and computational costs. Prompting methods such as Chain-of-Thought (CoT) (Wei et al. 2022) guide the model to generate step-by-step reasoning, but they do not impose explicit constraints on whether each reasoning step is supported by the input information. As a result, the generated reasoning

may appear coherent while still containing unsupported or incorrect steps. Solver-based methods introduce structured symbolic representations and external solvers, which can enforce formal correctness, but they often rely on task-specific representations and may reduce semantic transparency.

More importantly, existing approaches primarily aim to improve answer correctness, but they generally do not explicitly control when the model should refrain from answering. In high-stakes reasoning tasks, generating an unsupported answer may be more harmful than providing no answer at all. In practice, LLMs may still generate answers even when reliable reasoning support is insufficient.

Therefore, it is necessary to explore an approach that can explicitly constrain the reasoning process by introducing external validation mechanisms. In this context, "constraining the reasoning process" refers to enforcing that generated reasoning steps must be supported by the available information, and that answers should only be accepted when such support can be verified. This provides a way to improve reasoning reliability by filtering unsupported outputs, rather than relying solely on improving the model's internal reasoning capability.

1.2 Research Question

Existing research has explored multiple ways to improve the reasoning performance of LLMs, including prompting strategies and external tools. However, less attention has been paid to explicitly controlling the reasoning process, particularly ensuring that generated reasoning steps are supported by available information and determining when a model should refrain from answering.

This study addresses this gap by investigating whether external mechanisms can regulate the reasoning process without modifying the underlying model. Here, external mechanisms refer to auxiliary components that operate outside the model, such as structured representations of input information, reasoning validation procedures, and controlled revision or rejection steps.

Specifically, the study explores a validation-based framework that evaluates whether generated reasoning is supported by structured representations of the input information. Rather than focusing solely on answer accuracy, the study examines whether reasoning reliability can be improved through validation-based control of the reasoning process.

The Main Research Question:

Can the reasoning reliability of LLMs be improved, without retraining the model, by introducing external validation mechanisms that regulate the reasoning process?

Sub-Question:

How can such validation-based mechanisms be designed and integrated into the model's reasoning process to ensure that generated reasoning is supported by available information?

2

Extended Background

2.1 Limitations of Reasoning Capabilities of LLMs

Mainstream LLMs (such as GPT (Achiam et al. 2023), Claude (Anthropic 2024), and Gemini (Team et al. 2023)) are mostly based on the Transformer architecture (Vaswani et al. 2017).

The self-attention mechanism of the Transformer architecture enables models to generate outputs through context-based token prediction. As a result, reasoning still relies on prediction rather than explicit logical verification, which may produce responses that are semantically fluent but factually incorrect (Bender et al. 2021).

Researchers have evaluated the logical reasoning capabilities of LLMs from multiple perspectives. Xu et al. (2025) used various logical reasoning datasets to compare the performance of LLMs on deductive reasoning, inductive reasoning, abductive reasoning, and mixed-form reasoning tasks, and also provided brief descriptions of these types of reasoning in their paper:

- Deductive Reasoning: Starting from given premises and rules, conclusions are derived through logical inference. It is a top-down reasoning approach.
- Inductive Reasoning: General principles are derived from multiple specific observations. The conclusions are probabilistic rather than certain, and it is a bottom-up reasoning approach.

- Abductive Reasoning: Based on given observations, the most likely explanation or cause is inferred. The conclusions are also probabilistic and emphasize the "most plausible explanation".
- Mixed-form Reasoning: In complex scenarios, multiple types of reasoning (such as inductive reasoning followed by deductive reasoning) are combined to perform multi-step reasoning in order to obtain the final conclusion.

The experimental results of Xu et al. (2025) show that LLMs perform relatively well on deductive reasoning tasks, but perform poorly on other types of reasoning tasks. For example, Claude-3.5 can achieve an accuracy of up to 97% on deductive reasoning tasks, while its highest accuracy on abductive reasoning tasks is only 49%. Tyen et al. (2024) evaluated the ability of LLMs to detect reasoning errors. Among the evaluated models, GPT-4 achieved the best performance, but its highest overall accuracy was only 52.87%. Kamoi et al. (2024) compared the performance of LLMs and Expert Humans on the same error detection tasks. The results show that Expert Humans significantly outperform LLMs. Their F_1 scores in all task categories are higher than those of all evaluated LLMs. The highest F_1 score achieved by LLMs across all tasks is 90.8, while when Expert Humans perform the same tasks under the same evaluation settings, the F_1 score reaches 98.3.

Existing studies indicate that LLMs have clear limitations in logical reasoning capabilities. They may perform well on specific types of reasoning tasks, but cannot flexibly handle all types of reasoning tasks. Their ability to detect reasoning errors is limited and cannot reach the level of human experts.

2.2 Existing Approaches to Improving Reasoning Capabilities of LLMs

Fine-tuning methods aim to strengthen reasoning ability by training models on logically structured corpora (Morishita et al. 2024; Phan et al. 2023). Here, "logically structured corpora" refer to datasets that explicitly encode reasoning processes, such as step-by-step reasoning traces, annotated logical relations, or rule-based supervision signals. These datasets provide structured guidance for the model during training, enabling it to learn patterns of valid reasoning. Recent work has further explored error-driven

fine-tuning strategies, where models learn from previously identified reasoning failures to improve logical performance (Tong et al. 2024). However, fine-tuning requires retraining and does not guarantee that the model will follow valid reasoning steps during inference.

Prompt-based methods, such as CoT and its variants (e.g., Synthetic Prompting (Shao et al. 2023); Deductive Verification (Ling et al. 2023)), instruct models to generate intermediate reasoning steps through carefully crafted prompts. These methods improve interpretability by making reasoning processes more explicit. However, they lack explicit constraints in the sense that the generated reasoning steps are not systematically verified against the input information. As a result, the model may produce reasoning chains that appear coherent but are not fully supported by the given context.

Solver-based methods translate natural language problems into formal symbolic logic expressions and employ external solvers for verification (Di et al. 2025; Lyu et al. 2023). These approaches introduce explicit correctness checks through formal reasoning. However, they rely on abstract symbolic representations, which may reduce semantic interpretability and introduce additional errors during the translation process.

Overall, existing approaches either improve reasoning performance implicitly (through training or prompting) or introduce external verification in a formalized manner. However, they do not fully address reasoning regulation in the specific sense used in this study: validating whether generated reasoning steps are supported by the available input information and deciding whether the final answer should be accepted, revised, or rejected.

2.3 Negative Effects of Improving Reasoning Capabilities of LLMs

Although researchers have explored various methods to improve the reasoning capabilities of LLMs, existing approaches still have important limitations. In addition to the inherent limitations caused by their underlying principles, as mentioned in Section 1.1, these methods may also introduce undesirable effects.

Xu et al. (2025) compared the performance of LLMs when directly generating answers and when using CoT on the same reasoning datasets. The results show that CoT does not necessarily improve reasoning accuracy and

may even lead to a decrease in performance. For example, when Claude-3.5 directly processes the CLUTRR dataset (Sinha et al. 2019), its accuracy reaches 29%, whereas when CoT is used, the accuracy drops to 21%.

Kirichenko et al. (2025) found that although fine-tuning can improve the accuracy of LLMs' responses, it reduces the average abstention rate by 24%. This indicates that models become more likely to produce answers even when they are uncertain, rather than choosing to refrain from answering.

These findings reveal three key limitations of existing approaches. First, they do not provide explicit validation mechanisms to ensure that generated reasoning steps are supported by the input information. Second, they lack control over answer acceptance, meaning that models may still produce answers under uncertain or weakly supported conditions. Third, improvements in reasoning performance do not necessarily translate into improvements in reasoning reliability.

Therefore, it remains necessary to explore approaches that not only improve answer accuracy but also explicitly regulate the reasoning process and control when an answer should be accepted or rejected.

2.4 Graph-based Approaches for Reasoning and Reasoning Verification

Knowledge Graphs (KGs) are a method for representing entities and their relationships in a structured form, usually represented as a set of triples (h, r, t), where h and t denote entities and r denotes the relation between them (Chen et al. 2020). This structured representation organizes knowledge as a graph and supports reasoning through relation composition. Given a set of known facts, new relations can be inferred through multi-hop reasoning.

Graph-constrained Reasoning (GCR) (Luo et al. 2025) is a method that constrains the reasoning process of LLMs based on KGs, by restricting the model to reason along valid paths in the knowledge graph, thereby reducing the generation of incorrect reasoning results. Another related method is Graph of Thoughts (GoT) (Besta et al. 2024), which is an extension of the CoT method that models the reasoning process as a graph structure, where intermediate reasoning steps are represented as nodes and dependencies between them are represented as edges.

These methods use graph structures to organize or constrain the reasoning process. However, graph structures can not only be used to guide

reasoning generation, but also to verify the consistency of the reasoning process. KGs have also been used for information verification and anti-fraud tasks (Kapetanakis et al. 2012), where reliability is determined through relation consistency.

However, existing graph-based approaches mainly focus on structuring or guiding the reasoning process, rather than explicitly validating whether the generated reasoning is fully supported by the available information. In particular, they do not systematically verify reasoning steps against a structured representation of the input, nor do they explicitly control whether an answer should be accepted or rejected based on validation results.

This reveals a gap in current research: the lack of a validation-based mechanism that can regulate the reasoning process and determine whether a generated answer should be accepted, revised, or rejected based on its consistency with available information.

2.5 Reasoning Tasks and Datasets

The delexicalized (i.e., representations that do not rely on specific natural language semantic information) reasoning dataset NeuLR (Xu et al. 2025) and the classic kinship (family relationships) reasoning dataset CLUTRR (Sinha et al. 2019) can be used to evaluate the reasoning reliability of LLMs.

NeuLR contains deductive, inductive, and abductive reasoning tasks. Its delexicalized design reduces the influence of natural language semantics, allowing the evaluation to focus more directly on reasoning ability.

CLUTRR requires the model to infer kinship relations between entities based on natural language narratives. It involves implicit reasoning over family relations and requires the model to combine multiple pieces of information.

These datasets provide complementary evaluation settings. NeuLR enables controlled evaluation of different reasoning types, while CLUTRR reflects more complex reasoning scenarios expressed in natural language. Together, they allow a comprehensive assessment of reasoning reliability across different task structures.

3

Method

3.1 Research Strategy

This study adopts a quantitative research strategy to evaluate whether the proposed framework can improve the reasoning reliability of LLMs. Specifically, the study follows an experimental design, where the performance of the same model is compared under two conditions: with and without the proposed framework.

In this setting, the independent variable is the use of the framework, while the dependent variables are the model's performance metrics, including answer correctness rate, answer error rate, and refusal rate. These metrics are used to quantitatively assess the reasoning reliability of the model.

Quantitative research typically involves transforming research problems into measurable variables and analyzing them using statistical or computational techniques (Creswell et al. 2003). It is suitable for studies that aim to evaluate the effect of specific factors in a controlled setting (Bahari 2010). Since this study focuses on measuring the impact of an external mechanism on model behavior, a quantitative experimental approach is appropriate.

An alternative approach would be to use qualitative methods, such as interviews or user studies, to analyze how users perceive or interact with the framework. However, qualitative approaches focus on subjective experiences and textual data (such as interview answers) rather than measurable performance outcomes (Bryman 2003). Due to the objective of this

study and practical constraints, qualitative methods are not adopted. Future work may combine quantitative and qualitative approaches to further analyze user experience.

3.2 Research Method

This study proposes a validation-based reasoning control framework to improve the reasoning reliability of LLMs. Instead of modifying or retraining the underlying model, the framework introduces an external control layer that structures, evaluates, and regulates the reasoning process. The framework is designed to be model-agnostic and operates through prompt-based interaction with the LLM.

The proposed framework consists of three main components: (1) structured representation, (2) reasoning generation, and (3) validation-based control. Together, these components transform free-form reasoning into a structured process that can be explicitly evaluated and controlled.

3.2.1 Structured Representation

Given an input text T, the framework decomposes it into information I and a question Q, i.e., $T = I + Q$. The information I is then transformed into an information structure graph G_I, in which entities are represented as nodes and relations as edges. The question Q is later used to identify query-relevant entities during the construction of the question-focused graph G_Q for second-round reasoning. This structured representation is inspired by KGs, which enable explicit modeling of relational information.

In parallel, the reasoning process generated by the model is represented as a reasoning chain C, which is further transformed into a reasoning structure graph G_C. This dual representation allows both the input information and the reasoning process to be expressed in a comparable structured form.

This design builds on the general CoT prompting approach (Wei et al. 2022), which improves reasoning performance by encouraging step-by-step reasoning, and extends it by introducing an explicit structural representation of reasoning that can be externally verified.

3.2.2 Validation-Based Reasoning Control

The core of the proposed framework is a validation mechanism that evaluates whether the generated reasoning is supported by the available information. Unlike traditional approaches that rely solely on the model's internal reasoning capabilities, this framework introduces an explicit validation step to regulate the reasoning process.

At a general level, validation checks whether the reasoning structure graph G_C is supported by the information structure graph G_I. This idea is related to subgraph matching in graph-based systems (Angles and Gutierrez 2008; Cordella et al. 2004), where a candidate structure is verified against a larger graph to determine consistency. However, reasoning tasks differ in their underlying logical structure. Therefore, this study adopts a task-adaptive validation strategy, where the validation criteria are manually selected according to the predefined reasoning task type rather than automatically inferred by the framework itself. The validation criteria are then instantiated differently depending on the type of reasoning task:

- For tasks that require direct inference from explicitly stated relations, validation is based on structural consistency between G_C and G_I.
- For tasks that involve rule-based inference, validation allows reasoning steps that are supported through implicit logical closure derived from the information structure.
- For tasks that require identifying missing supporting facts, validation permits limited extensions of the information structure, provided that they do not conflict with existing information.

This design reflects the observation that a single uniform validation rule is insufficient for handling diverse reasoning paradigms. Instead, the framework provides a unified validation principle with task-specific instantiations.

3.2.3 Reasoning Revision and Refusal Mechanism

When the generated reasoning fails validation, the framework does not directly accept the output. Instead, it introduces a controlled reasoning process:

1. The model is guided to analyze the failure and produce an explanation.

2. A revised reasoning attempt is generated based on this feedback.

3. The revised reasoning is validated again.

If the reasoning still fails validation after revision, the framework explicitly rejects the answer and outputs a refusal.

This design treats refusal not as an error, but as a valid outcome of the reasoning process. By preventing unsupported or inconsistent answers from being produced, the framework improves the overall reliability of the system.

3.2.4 Relation to Existing Approaches

The proposed framework is related to several existing research directions. First, it follows the idea of CoT prompting (Wei et al. 2022) to guide the model to produce step-by-step reasoning, which is then used as the basis for subsequent validation.

Second, it is inspired by KG representations (Hogan et al. 2021), where entities and their relations are explicitly modeled as graph structures. This allows both input information and generated reasoning to be represented in a structured and comparable form.

Third, it is related to existing work on external verification mechanisms (Weng et al. 2023), where model outputs are checked against structured representations or constraints. However, unlike approaches that focus on improving answer accuracy through additional tools or modules, this study emphasizes using validation as a mechanism to regulate the reasoning process itself.

In contrast to these existing approaches, the framework in this study does not aim to enhance reasoning performance by introducing external solvers or optimizing generation directly. Instead, it focuses on structuring the reasoning process and applying validation mechanisms that determine whether a generated reasoning chain should be accepted, revised, or rejected.

In this sense, the method can be viewed as treating reasoning as a controlled process guided by validation, rather than relying solely on unconstrained generation.

3.2.5 Model Independence

The framework is designed to operate independently of the internal capabilities of the base model. Although modern LLMs may already be post-trained to improve reasoning performance, such improvements do not guarantee consistency or reliability. The proposed framework addresses this limitation by introducing an external validation layer that constrains the reasoning process regardless of the model's internal training.

As a result, the framework can be applied to different LLMs without requiring retraining, and its effectiveness does not depend on whether the model has been specifically optimized for reasoning tasks.

3.3 Data Collection Method

This study will use two existing reasoning datasets, NeuLR (Xu et al. 2025) and CLUTRR (Sinha et al. 2019), for evaluation.

NeuLR is a relatively small-scale delexicalized dataset, which evaluates the capability of LLMs to handle deductive, inductive, and abductive reasoning tasks in a neutral manner. Each type of reasoning task in NeuLR contains 1,000 samples. This study uses 3,000 samples to evaluate the reasoning reliability of the model on delexicalized deductive, inductive, and abductive reasoning tasks.

Figure 3.1, 3.2, and 3.3 present examples of the three types of NeuLR reasoning samples after preprocessing. The preprocessing step standardizes the dataset into a unified JSON structure containing only the fields *id*, *context*, *question*, and *label*. Auxiliary fields such as *explain*, which contain dataset-provided reasoning explanations or proof information, are removed to avoid exposing supporting reasoning paths to the model during evaluation. For the abductive setting, a standardized question ("What missing fact would support the given fact?") is additionally introduced to unify the input format across samples.

CLUTRR is a synthetic dataset for kinship reasoning. Here, "synthetic dataset" refers to a dataset in which reasoning task samples are automatically generated based on predefined rules. It was originally designed to evaluate the reasoning capability of Natural Language Understanding (NLU) models.

```
{
    "id": 1,
    "context": "SPmgoBnY are afraid of SPINyfQW. SPcqWqmr are
        afraid of SPINyfQW. NPWBE5eK is a SPcqWqmr. SPINyfQW
        are afraid of SP2Luin0s. SP2Luin0s are afraid of
        SPmgoBnY. NPkx5YGQ is a SPINyfQW. NPzIHHFV is a
        SP2Luin0. NPKK8F22 is a SPmgoBnY.",
    "question": "What is NPzIHHFV afraid of?",
    "label": "SPmgoBnY"
  }
```


Figure 3.1: Example of a Preprocessed NeuLR Deductive Reasoning Sample


```
  {
    "id": 1,
    "context": "NPLFtGrc is a SPymRS1n. NPQDgnvV is a SPDv96at.
         NPiPaofG is a SPymRS1n. NPQDgnvV is ADPkMYw3e.
        NPmNyqC3 is a SPDv96at. NPLFtGrc is ADPMaCnxM. NPlC2MoH
         is a SPLRmTBF. NPlC2MoH is ADPMaCnxM. NPiPaofG is
        ADPMaCnxM.",
    "question": "What property is NPmNyqC3?",
    "label": "ADPkMYw3e"
  }
```


Figure 3.2: Example of a Preprocessed NeuLR Inductive Reasoning Sample

CLUTRR is constructed based on a controllable task generation mechanism with predefined rules and includes two main evaluation settings: *Systematic Generalization*, which is used to evaluate the model's ability to generalize to unseen expression forms or combinations of relations; and *Robust Reasoning*, which is used to evaluate model performance under noisy conditions. These evaluation settings include multiple task types and are organized into training and test sets according to predefined configurations. In these settings, the test sets may contain both instances that are similar to those in the training set and instances that involve unseen reasoning depths.

CLUTRR represents relations among entities through natural language narratives and requires the model to infer kinship relations between entities. It has been shown to be applicable for evaluating the reasoning capability of LLMs. However, this study does not adopt its full task generation and training setup, and instead only uses pre-generated dataset subsets released by the original authors as evaluation data.

```
{
  "id": 0,
  "context": "Fact1: NP7NNSP4R6ytI is ADPRl020G.\nFact2:
      NP7NNSP4R6ytI is ADP3nnQNq.\nFact3: NPafp1fg is
      ADP5Gv8fK.\nFact4: NPafp1fg is ADPvmhKzP.\nFact5:
      NPhIh2ng is ADPL5ldlA.\nFact6: NPhIh2ng is ADPRl020G.\
      nFact7: NPdkK0fF is ADPzBtc2v.\nRule1: All ADPVKYvxg
      things are ADPL5ldlA.\nRule2: All ADPvmhKzP, ADP3nnQNq
      things are ADP5Gv8fK.\nRule3: If something is ADPL5ldlA
       then it is ADPvmhKzP.\nRule4: All ADPL5ldlA, ADP5Gv8fK
       things are ADPzBtc2v.\nRule5: If NPdkK0fF is ADPRl020G
       and NPdkK0fF is ADPL5ldlA then NPdkK0fF is ADP5Gv8fK.\
      nRule6: If something is ADPRl020G and ADP3nnQNq then it
       is ADPVKYvxg.\nRule7: If NPhIh2ng is ADP5Gv8fK and
      NPhIh2ng is ADPvmhKzP then NPhIh2ng is ADP3nnQNq.\
      nRule8: All ADPRl020G things are ADP3nnQNq.\nThe fact
      is: NPafp1fg is ADPVKYvxg.",
  "question": "What missing fact would support the given fact
      ?",
  "label": "NPafp1fg is ADPRl020G."
}
```


Figure 3.3: Example of a Preprocessed NeuLR Abductive Reasoning Sample

Specifically, this study uses the test samples (447 samples) from the dataset with the identifier *data_7c5b0e70* under the *Robust Reasoning* setting (hereafter referred to as *clutrr_mixed*). These samples include both noise-free and multiple types of noisy tasks (such as supporting facts, irrelevant facts, and disconnected facts), enabling the evaluation of the model's reasoning reliability when handling complex reasoning tasks described in natural language.

In addition, this study uses the test samples (1,048 samples) from the dataset with the identifier *data_db9b8f04* under the *Systematic Generalization* setting (hereafter referred to as *clutrr_clean*) for baseline evaluation.

Due to the high computational cost introduced by the framework, *clutrr_clean* is not included in the framework-based evaluation.

Therefore, *clutrr_mixed* serves as the primary CLUTRR setting for comparing baseline and framework performance.

Figure 3.4 presents an example of a preprocessed CLUTRR sample (from *clutrr_mixed*). The preprocessing step converts the original CSV-based dataset into a unified JSON format. The original *story* and *genders* fields are merged into a single *context* field, while the tuple-based *query* field is transformed

into a natural-language question. The target relation is stored as the *label*. In addition, auxiliary fields related to proof structures, graph edges, and dataset generation metadata are removed because they contain intermediate reasoning information that is not intended to be provided to the model during evaluation. Both *clutrr_mixed* and *clutrr_clean* consist of multiple CSV files, which are merged into a single complete JSON file after preprocessing, while the original source filename is retained in the *source_file* field.


```
{
  "id": "fc7ae67a-667a-4dc6-883b-14019359b22f",
  "context": "Story:\n[Kelly] and her mother [Ernest] made
      breakfast together. [Constance] and her husband [Ernest
      ] wanted a child badly\n\nGender information:\
      nConstance:female,Ernest:male,Kelly:female",
  "question": "What is the relationship of Kelly to Constance
      ?",
  "label": "daughter",
  "source_file": "1.2_test.csv"
}
```


Figure 3.4: Example of a Preprocessed CLUTRR Sample (clutrr_mixed)

Xu et al. (2025) evaluated both datasets and the results show that LLMs exhibit clear performance limitations on both NeuLR and CLUTRR tasks. This indicates that both are challenging and suitable evaluation datasets for LLMs. They also enable further comparison of model performance when handling delexicalized reasoning tasks versus tasks described in natural language.

This study initially considered using random sampling to equalize the number of samples from NeuLR and CLUTRR. However, NeuLR contains three types of reasoning tasks, while the CLUTRR test set is divided into multiple subsets based on different task types, each with substantially different sample sizes. Random sampling could therefore introduce distributional bias. As a result, NeuLR is used as the primary experimental dataset, while *clutrr_mixed* from CLUTRR is used as a supplementary evaluation setting.

3.4 Base Model

This study uses GPT-5 as the base model. GPT-5 is a Large Language Model developed by OpenAI based on the Transformer architecture. It belongs to

the GPT series and includes multiple versions, such as GPT-5 (Base), GPT-5 Mini, GPT-5 Nano, and GPT-5 Pro (Leon 2025).

This study selects GPT-5 (Base) (version gpt-5-2025-08-07) as the base model due to its strong general performance across tasks. In addition, its maximum context length is 400k tokens and the maximum output length is 128k tokens, which is sufficient for the evaluation experiments.

Although commonly used models in previous research (such as GPT-4) were considered for comparison, this study focuses on GPT-5 due to its strong overall performance and wider practical usage (Leon 2025). Due to resource constraints, only one base model is used.

3.5 Data Analysis Method

This study adopts a quantitative analysis method to evaluate the experimental results. Although the tasks can be framed as classification problems, the primary focus of this study is not on label prediction accuracy alone, but on the model's reasoning behavior, including whether it produces a correct answer, an incorrect answer, or explicitly refuses to answer. Therefore, traditional classification metrics (such as F_1 score, precision, and recall) are not well aligned with the objectives of this study (Tharwat 2021).

In the evaluation experiments, when the model processes each sample, there are two possible behaviors: providing an answer or refusing to answer. When the model provides an answer, the result may be either correct or incorrect. Therefore, with reference to the metric "Answer Correctness" proposed in Xu et al. (2025), which evaluates tasks in the same domain using the same datasets, this study adopts the following three metrics to measure the reasoning reliability of the model. These metrics correspond to the dependent variables defined in Section 3.1:

- Answer correctness rate: the proportion of correctly answered questions out of the total number of questions, where a "correct answer" means that the answer provided by the model is consistent with the ground truth.
- Answer error rate: the proportion of incorrectly answered questions out of the total number of questions, where an "incorrect answer" means that the answer provided by the model is inconsistent with the ground truth.

- Refusal rate: the proportion of questions for which the model refuses to answer out of the total number of questions, where "refusal" means that the model outputs statements explicitly indicating that it cannot answer the question (such as "cannot answer this question").

Let N be the total number of samples, and let N_{correct}, N_{error}, and N_{refusal} denote the number of correct answers, incorrect answers, and refusals, respectively. The metrics are defined as:

- Answer correctness rate = N_{correct}/N
- Answer error rate = N_{error}/N
- Refusal rate = N_{refusal}/N

These three cases are mutually exclusive and cover all possible output behaviors of the model. Therefore, the sum of the three metrics is equal to 1, which is used to comprehensively describe the model's performance under different output behaviors.

3.6 Framework Implementation

This section describes how the proposed validation-based reasoning control framework is instantiated in practice, including the implementation of structured representations, task-specific validation mechanisms, and the overall experimental pipeline.

In simple terms, the framework first converts both the input information and the model-generated reasoning into graph structures. It then checks whether the reasoning is supported by the available information using different validation strategies for different reasoning tasks. If the reasoning fails validation, the model is asked to revise its reasoning once. If validation still fails, the framework outputs a refusal instead of returning an unsupported answer.

Figure 3.5 illustrates the overall reasoning and validation process of the proposed framework.

The implementation code, preprocessing scripts, and evaluation pipeline used in this study are publicly available in a GitHub repository to support reproducibility of the experiments: `https://github.com/biwu3994/validation-based-llm-reasoning`

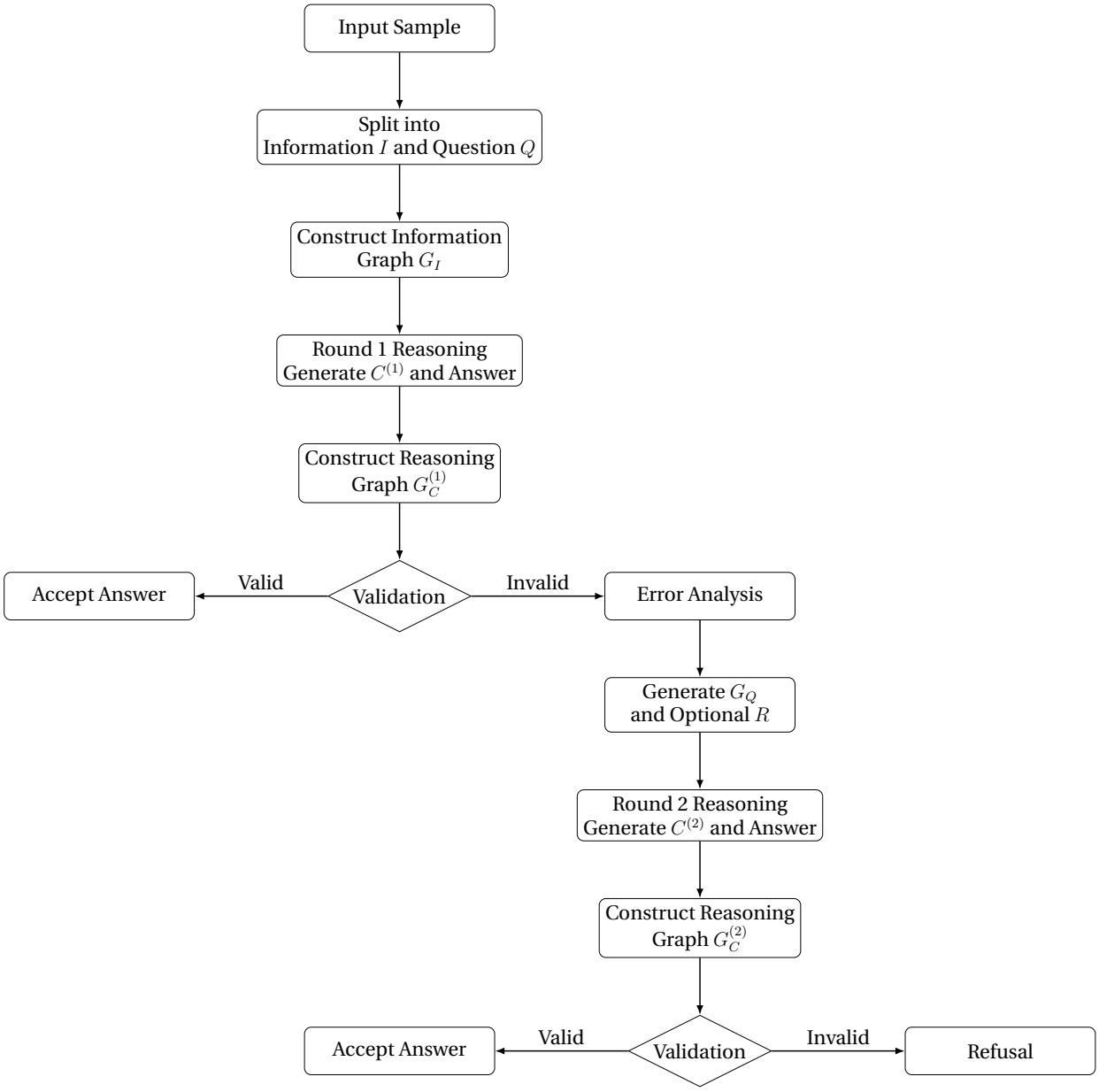


Figure 3.5: Overall pipeline of the proposed validation-based reasoning control framework.

3.6.1 Information and Reasoning Representation

The framework represents both input information and model-generated reasoning as graphs, inspired by graph-based knowledge representation approaches such as KGs and relational reasoning structures (Hogan et al. 2021; Ji et al. 2021).

Given an input sample, it is first separated into information I and question Q. The information I is then transformed into an information structure graph G_I, defined as:

- $G_I = (V, E)$, where
  - V is the set of entities (including both ordinary entities and rule nodes)
  - E is the set of directed edges of the form $(source, relation, target)$

This graph formulation follows the common node–edge representation used in graph-based knowledge representation, where entities are modeled as nodes and relations are modeled as directed edges (Hogan et al. 2021).

In implementation, a graph is stored as a set of nodes and edges. For example:

```
{
  "nodes": ["Kelly", "Constance"],
  "edges": [
    {
      "source": "Kelly",
      "relation": "daughter",
      "target": "Constance"
    }
  ]
}
```

This means that `Kelly` and `Constance` are graph nodes, and the edge represents the relation `daughter` from `Kelly` to `Constance`.

Two types of edges are included:

1. Fact edges, representing explicit relations between entities in the input information. These edges follow the standard form:

$$(entity_1, relation, entity_2)$$

2. Rule edges, representing logical rules explicitly stated in the input.

Each rule is modeled as a dedicated node $Rule_k$, where k is an index identifying a specific rule.

Instead of encoding a rule as a single structure, it is decomposed into multiple edges:

- $(Rule_k, \textit{premise}, \textit{predicate})$
- $(Rule_k, \textit{conclusion}, \textit{predicate})$

Here, *predicate* refers to expressions such as `is a(X,Class)` or `are afraid of(X,Y)`.

The premise edges represent the conditions of the rule, while the conclusion edges represent the inferred outcome. This design allows logical rules with multiple conditions to be represented in graph form. For example, a rule may contain multiple premise conditions that together support a conclusion.

Such rules are represented as a set of graph edges connected to the same rule node $Rule_k$.

This representation allows rules to be treated as first-class structures in the graph, rather than being implicitly encoded in text. Similar graph-based representations have been used in symbolic and neuro-symbolic reasoning systems to support explicit relational inference and rule-based reasoning (Garcez and Lamb 2023).

The reasoning process generated by the model is represented as a reasoning chain:

$$C = [s_1, s_2, ..., s_n]$$

Here, C represents the complete reasoning chain, while s_1 to s_n represent individual typed reasoning steps in sequential order.

Each step s_i belongs to one of the following step types:

- `given_fact`: a fact directly derived from the input information
- `rule`: a rule applied during reasoning
- `pattern`: an abstract pattern used in inductive tasks
- `candidate_answer`: the final inferred answer

Each step is expressed in a predicate-like form (e.g., `relation(Entity1,Entity2)`), so that it can be consistently mapped to graph edges.

In implementation, the reasoning chain is represented as a sequence of typed steps. For example:

```
{
  "type": "given_fact",
  "expr": "mother(Ernest,Kelly)"
}
```

or

```
{
  "type": "candidate_answer",
  "expr": "daughter(Kelly,Constance)"
}
```

The reasoning chain is then transformed into a reasoning structure graph G_C using the same edge schema as G_I. For example, the candidate-answer step above can be transformed into the following graph edge:

```
{
  "source": "Kelly",
  "relation": "daughter",
  "target": "Constance",
  "role": "candidate_answer"
}
```

This ensures that the reasoning output and the original information are represented in a unified format, enabling direct structural comparison during validation.

All auxiliary graphs in the framework, including the deductive closure graph, pattern graph G_P, question-focused graph G_Q, and expanded validation graph G_{I+R}, use the same underlying graph schema as G_I. However, they serve different semantic purposes during validation and reasoning control.

For NeuLR inductive tasks, the information structure graph G_I is constructed using rule-based extraction rather than LLM-based graph generation. This is because the NeuLR inductive samples follow consistent sentence patterns, where sentences containing "is a" indicate category membership, while sentences containing "is" without "a" indicate property assignment. This rule-based extraction reduces graph construction errors in this specific task setting.

3.6.2 Two-Round Reasoning Pipeline

The framework follows a two-round reasoning process. In the first round, the model generates reasoning and an answer. The framework then checks whether the reasoning is supported by the structured information. If validation fails, the model receives feedback and performs a second reasoning

attempt. If the revised reasoning still fails validation, the framework refuses to answer.

In the notation used below, the superscript (1) refers to the first reasoning round, and the superscript (2) refers to the second reasoning round. Therefore, $C^{(1)}$ means the first-round reasoning chain, while $C^{(2)}$ means the second-round reasoning chain. Similarly, $G_C^{(1)}$ means the reasoning structure graph constructed from the first-round reasoning chain, while $G_C^{(2)}$ means the reasoning structure graph constructed from the second-round reasoning chain.

1. First-round reasoning
    - The model generates a reasoning chain $C^{(1)}$ and an answer.
    - The reasoning chain $C^{(1)}$ is transformed into a reasoning structure graph $G_C^{(1)}$.
    - $G_C^{(1)}$ is validated against the corresponding validation graph.
2. Second-round reasoning (if needed)
    - If validation fails, the model is prompted with:
        - an error explanation ϵ, where ϵ denotes feedback explaining why the first-round reasoning failed validation
        - a question-focused graph G_Q
    - A revised reasoning chain $C^{(2)}$ is generated.
    - $C^{(2)}$ is transformed into $G_C^{(2)}$ and validated again.

If the second-round reasoning still fails validation, the system outputs a refusal.

3.6.3 Task-Adaptive Validation Mechanisms

Validation is implemented differently depending on the reasoning task. Instead of using a single uniform rule, the framework applies task-specific validation strategies.

In simple terms:

- Structural matching checks whether the reasoning structure graph is directly supported by the information structure graph.
- Deductive reasoning checks whether conclusions logically follow from known relations.

- Inductive reasoning checks whether predicted properties match observed category patterns.
- Abductive reasoning allows one missing fact if it can plausibly explain the observations.
- If unsupported reasoning remains after revision, the framework refuses to answer.

1. Structural Matching (General Case)

   For tasks based on explicit relations, validation first checks whether the reasoning structure graph is supported by the validation graph.

   In the strict structural setting, the reasoning structure graph is expected to be structurally supported by the information structure graph.

   In implementation, this means that each non-rule edge in G_C must be supported by an edge in G_I, and that rule structures, represented through premise–conclusion pairs, must be consistent with those in G_I. This formulation is related to graph-based structural matching and consistency checking, where relational structures are validated against a supporting graph representation (Angles and Gutierrez 2008).

   This validation strategy is used as the default structural validation mechanism before task-specific relaxation or alternative validation is applied.

2. Deductive Validation via Closure

   For deductive reasoning tasks, direct subgraph matching is insufficient because valid conclusions may not be explicitly stated in G_I.

   To address this, a deductive closure G_I^+ is constructed from G_I by propagating class-level relations to instance-level entities. Here, deductive closure means an expanded version of the information graph that also includes logically inferred relations derived from the original information.

   Formally, consider two types of edges in G_I:

   - instance–class membership edges of the form $(x, \textit{is a}, c)$, where x is an entity and c is a class
   - class-level relation edges of the form (c, r, y), where c is a class, r is a relation, and y is a target entity

If both relations exist in G_I, where entity x belongs to class c and class c has relation r with entity y, an additional inferred edge can be introduced:

$$(x, r, y)$$

Intuitively, this means that if an entity x belongs to a class c, and the class c has a relation r with another entity y, then the same relation can be propagated to the instance level.

The closure graph G_I^+ is obtained by adding all such inferred edges to G_I.

In implementation, deductive validation does not require the entire reasoning structure graph G_C to be a strict subgraph of G_I. Instead, the framework checks whether the candidate-answer edge in G_C is supported by the deductive closure graph G_I^+ after deductive inference has been applied.

This validation strategy allows implicit logical consequences to be accepted without requiring them to be explicitly stated in the original graph.

3. Inductive Validation via Pattern Graph

   For inductive reasoning tasks, the framework extracts category–property patterns from G_I.

   In this setting, two types of relations are distinguished:

   - category relations of the form $(e, \textit{category}, k)$, indicating that an entity e belongs to a category k
   - property relations of the form $(e, \textit{property}, p)$, indicating that the same entity e has a property p

   Here, k denotes a category label used for grouping entities in inductive reasoning, and p denotes a property or attribute value associated with an entity.

   Category variables used in inductive reasoning are distinct from class variables used in deductive reasoning. A class variable c participates in logical inference through class-level relation propagation, while a category variable k serves as a grouping label for extracting category–property patterns from data.

If an entity e is associated with both a category relation and a property relation in G_I, this means that entity e belongs to category k and has property p.

In this case, the framework constructs a pattern edge of the form:

$$(k, \textit{maps_to}, p)$$

Intuitively, this represents an observed association between a category k and a property p, derived from instances in G_I.

All such edges form a pattern graph G_P.

During validation, for a query entity e_q, the framework first identifies its category k from G_I, and then checks whether the predicted property p is supported by a corresponding category–property pattern in G_P.

This validation strategy allows the model to validate answers based on category–property regularities, rather than requiring the target property to be explicitly stated in G_I.

4. Abductive Validation with Missing Fact Constraint

   For abductive reasoning tasks, the goal is to infer a missing supporting fact.

   The strict structural matching requirement is relaxed under the constraint that:

   - exactly one unsupported edge is allowed
   - the unsupported edge must correspond to the candidate answer
   - the final answer must match the candidate edge in surface form
   - the edge must not conflict with existing information in the validation graph

   Conflict detection is performed by asking the model to determine whether the candidate edge explicitly contradicts the validation graph. This conflict check is only used for this abductive relaxation setting.

   If these conditions are satisfied, the reasoning is accepted as a valid missing-fact explanation. If the candidate edge conflicts with the validation graph, the reasoning is rejected.

3.6.4 External Relation Expansion

External relation expansion is used only when the first-round reasoning result fails validation and the task requires additional relational support that is not explicitly represented in the original information structure graph G_I.

In the implementation, this mechanism is mainly applied to CLUTRR. Since CLUTRR requires kinship composition, the original graph G_I may contain explicit family relations and gender information, but may not directly contain the queried kinship relation. In such cases, the framework introduces an auxiliary relation set R generated by the model.

The set R is represented using the same edge schema as G_I:

$$(source, relation, target)$$

It is not an independent knowledge graph, nor a full reasoning chain. Instead, it contains a limited number of additional relation edges that are relevant to the current question.

The expanded validation graph G_{I+R} is constructed by combining the original information structure graph G_I with the additional relation set R. As a result, G_{I+R} contains both the original relations and the newly added relations generated during relation expansion.

The generation of R is constrained to:

- include only relations relevant to the current question;
- use entities already present in G_I where possible;
- remain consistent with the existing information in G_I;
- avoid unrelated external knowledge.

In the CLUTRR setting, R is used to add implicit kinship relations derived from existing family relations and gender information. For example, if the original graph contains relations that imply a parent–child relation but the queried relation is not directly present, R may add the inferred kinship edge in the query direction.

This mechanism is not applied uniformly to all tasks. For NeuLR deductive reasoning, relation expansion is explicitly disabled because validation is handled through deductive closure. For NeuLR inductive reasoning, validation is based on the pattern graph G_P, so R is not used. For NeuLR abductive reasoning, relation expansion is not used when the validation failure is caused by an unsupported candidate-answer edge, because the

task itself requires identifying the missing fact rather than adding it to the validation graph.

When R is generated, the expanded graph G_{I+R} is used as the validation base for the second-round reasoning process.

3.6.5 Question-Focused Graph

To improve reasoning stability in the second round, a question-focused graph G_Q is constructed from the validation graph. The validation graph can be either G_I or G_{I+R}, depending on whether external relation expansion has been applied.

G_Q is intended to be a subgraph that retains:

- edges involving query-relevant entities
- minimal supporting edges required for reasoning

The query-relevant entities are first extracted from the question Q and context. Edges involving these entities are treated as mandatory edges. The model is then prompted to construct a focused graph while keeping these mandatory edges and avoiding invented information.

After generation, G_Q is filtered to ensure that all retained edges are actually present in the validation graph. If no valid focused subgraph can be constructed, the framework falls back to using the full validation graph as G_Q.

When a valid focused graph can be constructed, this reduces irrelevant information and constrains the second-round reasoning process.

3.6.6 Refusal Detection

The framework includes an explicit refusal mechanism.

A response is classified as a direct refusal if it contains expressions indicating inability to answer, such as `cannot answer`, `not enough information`, or `cannot determine`.

Refusal can occur in three ways:

- the model directly refuses in the first reasoning round
- the model directly refuses in the second reasoning round
- the second-round reasoning fails validation after correction

In the abductive validation setting, refusal is also enforced if the candidate missing fact conflicts with the validation graph.

3.7 Experimental Setup

The framework is evaluated under two conditions:

1. Baseline condition: the model generates answers without the framework
2. Framework condition: the full validation-based reasoning control pipeline is applied

For datasets evaluated under both conditions, the same samples are used in the baseline and framework settings, allowing direct comparison.

Before evaluation, predictions and gold labels are normalized by:

- lowercasing
- removing a final period
- removing the article “the”
- normalizing whitespace

Correctness is determined by exact match after normalization.

The evaluation uses the datasets described in Section 3.3 and the base model specified in Section 3.4.

Performance is measured using the metrics defined in Section 3.5.

3.8 Ethical Considerations

This study does not involve human participants, personal data, or sensitive information. All experiments are conducted using publicly available reasoning datasets and API-based LLM outputs.

The datasets used in this study, including NeuLR and CLUTRR, are research datasets designed for reasoning evaluation.

Since the proposed framework introduces external validation and refusal mechanisms, the system may refuse unsupported answers instead of generating potentially unreliable outputs. This design is intended to reduce unsupported reasoning generation during evaluation. However, the framework is evaluated only in a controlled experimental setting and is not intended for direct deployment in high-stakes real-world applications.

4

Results

4.1 Baseline Experiment Results

The baseline results on the NeuLR and CLUTRR datasets are summarized in Table 4.1. Overall, the model achieves the strongest performance on the NeuLR abductive reasoning task, while the lowest correctness rate is observed on the *clutrr_mixed* dataset. Refusal behavior is generally limited under the baseline setting, except for *clutrr_clean*, which shows a relatively higher refusal rate.

For the NeuLR abductive reasoning dataset, some correct predictions were initially misclassified as errors due to overly strict evaluation criteria. The results were therefore re-evaluated using a post-processing script, and the corrected results are reported here.

4.2 Framework Experiment Results

The framework results on the NeuLR and CLUTRR datasets are summarized in Table 4.1. Under the framework setting, the best performance is observed on the NeuLR inductive reasoning task, while *clutrr_mixed* remains the most challenging setting. Compared with the baseline condition, refusal behavior becomes substantially more frequent in some tasks, especially in *clutrr_mixed*.

For the NeuLR deductive reasoning dataset, inconsistencies in the initial evaluation criteria required re-evaluation, and the corrected results are reported here.

As described in Section 3.3, the *clutrr_clean* dataset is not evaluated under the framework setting due to the high computational cost introduced by the multi-stage reasoning pipeline.

4.3 Comparative Results

The results for both baseline and framework settings are summarized in Table 4.1. Overall, the framework consistently reduces the answer error rate across all evaluated datasets. However, its impact on answer correctness varies across reasoning tasks.

The NeuLR inductive and abductive reasoning tasks show improvements in correctness after applying the framework, whereas the NeuLR deductive task and *clutrr_mixed* show decreases in correctness. In addition, refusal behavior generally increases after applying the framework, with the largest increase observed in *clutrr_mixed*.

Figures 4.1–4.3 provide a visual comparison of correctness rates, error rates, and refusal rates between the baseline and framework settings across datasets.

Table 4.1: Overall Results for Baseline and Framework Settings

Dataset	Setting	Correct	Error	Refusal	Correctness Rate	Error Rate	Refusal Rate
NeuLR-Deductive	Baseline	788	212	0	0.788	0.212	0.000
NeuLR-Deductive	Framework	730	131	139	0.730	0.131	0.139
NeuLR-Inductive	Baseline	771	219	10	0.771	0.219	0.010
NeuLR-Inductive	**Framework**	**972**	**28**	**0**	**0.972**	**0.028**	**0.000**
NeuLR-Abductive	Baseline	856	144	0	0.856	0.144	0.000
NeuLR-Abductive	**Framework**	**903**	**90**	**7**	**0.903**	**0.090**	**0.007**
clutrr_mixed	Baseline	237	207	3	0.530	0.463	0.007
clutrr_mixed	Framework	138	148	161	0.309	0.331	0.360
clutrr_clean	Baseline	687	291	70	0.656	0.278	0.067

4.4 Summary of Results

Under the baseline condition, the model performs best on the NeuLR abductive reasoning dataset and worst on the *clutrr_mixed* dataset.

After applying the framework, the best performance is observed on the NeuLR inductive reasoning dataset, while *clutrr_mixed* remains the most challenging setting.

Across all datasets evaluated under both baseline and framework conditions, the answer error rate decreases after applying the framework. However, the impact on correctness rate varies across tasks. The NeuLR inductive and abductive tasks show clear improvements in correctness, whereas the NeuLR deductive task and *clutrr_mixed* show a decline in correctness.

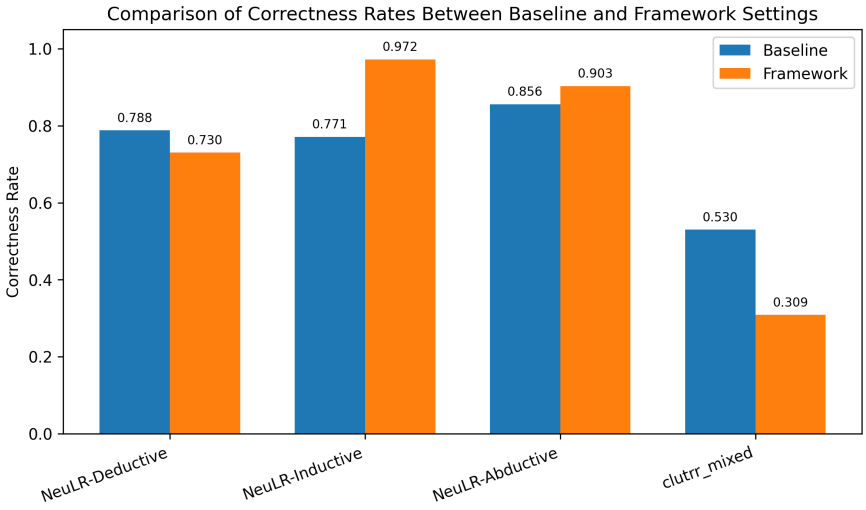


Figure 4.1: Comparison of Correctness Rates Between Baseline and Framework Settings

In terms of refusal behavior, only the NeuLR inductive dataset shows a reduction in refusal rate, while all other datasets exhibit an increase in refusal rate after applying the framework.

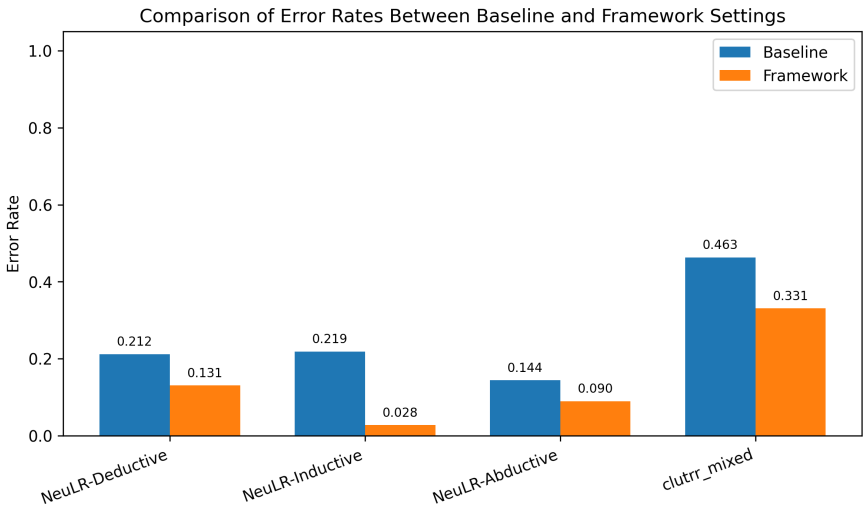


Figure 4.2: Comparison of Error Rates Between Baseline and Framework Settings

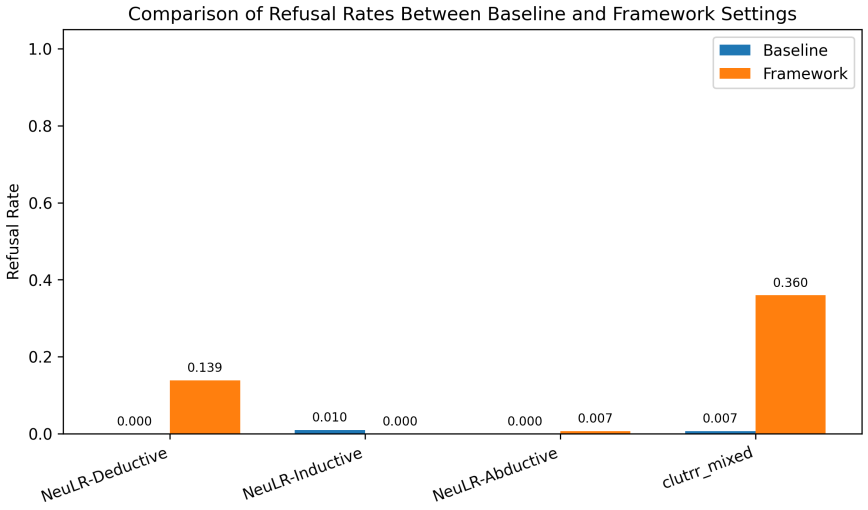


Figure 4.3: Comparison of Refusal Rates Between Baseline and Framework Settings

5

Analysis

5.1 Overall Effect of the Framework

The comparison between baseline and framework settings shows that the proposed framework consistently shifts model behavior toward reliability-oriented reasoning.

Across all datasets evaluated under both baseline and framework conditions, the answer error rate decreases, while the refusal rate generally increases, except for the NeuLR inductive task. This indicates that unsupported outputs are filtered and replaced by explicit refusals rather than being returned as answers.

The impact on answer correctness rate is task-dependent. For NeuLR inductive and abductive reasoning, the correctness rate increases, while for NeuLR deductive reasoning and CLUTRR, it decreases.

These results indicate that the proposed framework does not uniformly improve accuracy, but instead encourages more conservative and reliability-oriented reasoning behavior.

5.2 Task-wise Analysis

5.2.1 Deductive Reasoning

For the NeuLR deductive reasoning task, the framework reduces the answer error rate but also decreases the answer correctness rate, while introducing a substantial refusal rate.

First, a portion of the errors in the baseline setting originates from surface-form mismatches rather than incorrect reasoning. For example, the model may generate a plural class label (e.g., *SPIf3r6Bs*) while the gold label is its singular form (*SPIf3r6B*) (both are delexicalized labels from the original dataset). Although the reasoning is structurally valid and supported by the deductive closure graph G_I^+, such answers are still counted as errors due to exact-match evaluation. The framework includes limited normalization for such surface-form variation, but it does not fully eliminate this type of mismatch.

Second, the validation mechanism requires the candidate answer to be supported by the deductive closure graph G_I^+. As a result, unsupported inferences, such as incorrectly propagating a class-level relation to an instance without explicit support, are rejected during validation.

Finally, when the reasoning chain fails validation and cannot be corrected in the second round, the framework outputs a refusal instead of an answer. This explains the significant increase in the refusal rate.

Overall, the framework improves reliability through stricter structural validation, but at the cost of reduced coverage.

5.2.2 Inductive Reasoning

For the NeuLR inductive reasoning task, the framework significantly improves the answer correctness rate while eliminating refusals entirely. However, a portion of errors still remains.

These errors are mainly caused by ambiguity in pattern-based inference, where multiple properties may be associated with the same category in G_P. As a result, the model may generate a property that is supported by the pattern graph but does not correspond to the gold label. Since the candidate property already satisfies the validation condition, no second-round correction is triggered.

Another notable observation is that no refusals are produced in the inductive task. Since G_P is derived from G_I, a supporting pattern can almost always be found for a given query. Therefore, the rejection mechanism is rarely activated.

Overall, inductive validation achieves high coverage because it only enforces pattern consistency rather than strict structural constraints.

5.2.3 Abductive Reasoning

For the NeuLR abductive reasoning task, the framework increases the answer correctness rate and reduces the answer error rate, while introducing only a small refusal rate.

First, some errors are caused by ambiguity in missing-fact generation. The framework accepts a candidate missing fact if it corresponds to the candidate answer and does not explicitly conflict with G_I. However, several different missing facts may be structurally compatible with the same information graph. As a result, a plausible but incorrect hypothesis may still pass validation.

Second, refusals occur when the reasoning graph contains more than one unsupported edge. The release mechanism only accepts reasoning graphs with a single unsupported candidate-answer edge. If additional unsupported rule structures appear in the reasoning graph, the entire reasoning process fails validation.

This problem is especially visible when the model uses unstable predicate forms or variable names, such as using `something` instead of the variable form used in G_I. Although these expressions may be semantically similar, they are treated as different structures during graph matching.

Overall, the abductive setting shows that the framework can constrain unsupported missing-fact generation, but it cannot fully resolve hypothesis ambiguity.

5.2.4 CLUTRR

The CLUTRR task shows the weakest framework performance among the evaluated datasets. After applying the framework, the answer correctness rate on *clutrr_mixed* decreases substantially, while the refusal rate increases sharply.

A major reason is that CLUTRR requires implicit kinship composition. Unlike NeuLR tasks, where the reasoning structure is more directly encoded in the input, CLUTRR often requires kinship relations to be inferred from family relations, gender information, and natural-language story context. Therefore, when first-round reasoning fails validation, the framework performs external relation expansion by constructing an additional relation set R.

However, relation expansion itself becomes a major source of errors. If R introduces an incorrect kinship relation, the second-round reasoning graph may still match the expanded validation graph G_{I+R} structurally

even though the final answer is semantically incorrect. This reveals a limitation of the framework: structural consistency does not guarantee semantic correctness of the expanded graph itself.

Another problem is representation mismatch. Some generated kinship relations use different relation forms, such as `daughter of` instead of `daughter`. Although these forms are semantically similar, the validation process relies on normalized graph-edge matching. As a result, structurally plausible reasoning may still fail validation and lead to refusal.

There are also cases where the model directly refuses in the second round even when R contains potentially useful relations. This occurs when the model fails to use the added relation correctly in the reasoning graph or when the question-focused graph emphasizes an incomplete relation path.

Overall, the framework is less effective for CLUTRR because the task depends heavily on implicit relation expansion and flexible natural-language interpretation, both of which are difficult to represent consistently in graph form.

5.2.5 Summary of Task-wise Analysis

The task-wise analysis shows that the framework performs best when reasoning structures can be reliably converted into graph-based validation conditions. NeuLR inductive and abductive reasoning benefit from relatively stable structural patterns, whereas CLUTRR remains difficult because it depends on implicit kinship composition and flexible natural-language relation representation. Overall, the effectiveness of the framework is closely tied to how consistently the reasoning process can be externalized into structured graph representations that support reliable validation.

6

Discussion

6.1 Comparison with Previous Research

Xu et al. (2025) evaluated ChatGPT on the NeuLR reasoning tasks. For CLUTRR, they evaluated both ChatGPT and GPT-4. However, direct comparison should be interpreted cautiously because the experiments use different settings. In addition, Xu et al. (2025) reported results as percentages, while this study reports correctness rates in decimal form.

For the NeuLR deductive reasoning task, Xu et al. (2025) reported that ChatGPT achieved 85.20% correctness under zero-shot prompting and 68.30% after CoT prompting. In this study, GPT-5 achieves 78.8% under the baseline setting and 73.0% after applying the framework. A similar decrease is therefore observed in both studies. However, the framework-based GPT-5 result remains higher than the CoT-based ChatGPT result reported by Xu et al. (2025).

For the NeuLR inductive reasoning task, Xu et al. (2025) reported that ChatGPT improved from 15.10% to 69.60% after CoT prompting. A similar improvement pattern is observed in this study, where GPT-5 improves from 77.1% under the baseline setting to 97.2% after applying the framework. This suggests that both CoT prompting and the proposed framework are particularly beneficial for inductive reasoning tasks, although the framework-based result remains substantially higher than the CoT-based ChatGPT result reported by Xu et al. (2025).

For the NeuLR abductive reasoning task, Xu et al. (2025) reported that ChatGPT decreased from 11.50% to 6.10% after CoT prompting. In contrast, GPT-5 improves from 85.6% under the baseline setting to 90.3% after applying the framework. Unlike the CoT-based setting reported by Xu et al. (2025), the proposed framework does not reduce abductive reasoning performance. Instead, the abductive validation mechanism appears to constrain unsupported missing-fact generation while still allowing one plausible missing fact to be introduced.

Xu et al. (2025) also evaluated ChatGPT and GPT-4 on CLUTRR tasks. Direct comparison should be interpreted cautiously because different CLUTRR configurations may have been used. Nevertheless, similar trends can be observed. Xu et al. (2025) reported that ChatGPT decreased from 21.99% under zero-shot prompting to 19.55% and 12.83% under one-shot and three-shot prompting, respectively. GPT-4 also decreased from 33.00% to 26.00% after CoT prompting. Similarly, applying the proposed framework to GPT-5 reduces correctness on the *clutrr_mixed* dataset.

Overall, the comparison with Xu et al. (2025) suggests that external reasoning assistance does not uniformly improve reasoning performance across tasks. Both studies show that inductive reasoning benefits substantially from additional reasoning support, whereas deductive and CLUTRR-style kinship reasoning may experience performance degradation. Unlike the CoT-based setting reported by Xu et al. (2025), however, the proposed framework improves abductive reasoning correctness while reducing the answer error rate. This suggests that validation-based reasoning control may behave differently from standard CoT prompting by explicitly regulating whether generated reasoning is structurally supported by the available information.

6.2 Overall Discussion

The results of this study suggest that the answer to the main research question is yes: the reasoning reliability of LLMs can be improved without retraining by introducing external validation mechanisms that regulate the reasoning process. However, the improvement mainly appears as a reduction in unsupported answers rather than a uniform correctness increase. The results also suggest that task-adaptive validation mechanisms are important, since different reasoning tasks require different forms of structural support and validation criteria.

Regarding the sub-question, the results suggest that such validation-based mechanisms can be designed by transforming both input information and generated reasoning into structured graph representations and applying task-adaptive validation rules to evaluate whether the reasoning is supported by the available information structure. This effect is achieved through validation-based control, which filters out reasoning results that cannot be structurally supported by the constructed graph representations. As a result, the model becomes more conservative and produces fewer incorrect answers, but also increases the number of refusals.

The effectiveness of the framework depends on the alignment between the validation mechanism and the structure of the reasoning task. When the reasoning process can be clearly represented and verified, as in inductive and abductive tasks, the framework can improve both correctness and reliability. In contrast, when reasoning relies on implicit composition or flexible natural-language interpretation, as in CLUTRR, the framework becomes less effective.

Overall, the proposed approach demonstrates that external validation mechanisms can be used to control reasoning behavior without modifying the underlying model. However, its performance is constrained by the quality of graph construction, relation representation, and task-specific validation design.

6.3 Computational Cost

A practical limitation of the proposed framework is its computational cost. The framework requires multiple additional steps, including graph construction, reasoning generation, validation, relation expansion, and second-round reasoning, resulting in substantially higher token consumption and runtime overhead than baseline inference.

The experiments consumed more than ten million tokens and required over 16,000 API requests, with a total cost exceeding 290 USD according to OpenAI API usage records.

The computational overhead also affected experimental feasibility. For example, although the *clutrr_clean* subset was used in the baseline evaluation, it was excluded from the framework-based experiments due to the cost and runtime requirements of the multi-stage pipeline.

While the framework improves reasoning reliability, it does so at the expense of efficiency, highlighting the need to balance reliability and computational cost in practical applications.

6.4 Societal Consequences

As LLMs are increasingly applied in areas such as education, healthcare, and decision support, unsupported or unreliable reasoning may lead to misinformation or incorrect decisions. The proposed framework aims to reduce such risks by introducing external validation mechanisms that encourage more reliable reasoning behavior. However, the framework also increases refusal behavior and computational cost, which may affect usability and accessibility in practical applications. Therefore, balancing reliability, transparency, and efficiency remains an important societal challenge for future LLM systems.

6.5 Future Work

Future work may evaluate the framework on additional LLMs, more realistic datasets, and improved validation mechanisms. While the current study focuses on a single model and controlled reasoning tasks, future evaluations on diverse models and real-world datasets would provide a broader assessment of the framework's effectiveness. In addition, more robust relation expansion strategies, alternative validation designs, and the use of multiple external models may further improve reasoning verification.

6.6 Use of AI Tools

AI-based tools were used for grammar checking and minor language editing. All research ideas, system design, data analysis, and conclusions were developed by the author, who takes full responsibility for the content of this thesis.

Bibliography

Achiam, J., S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. (2023). "Gpt-4 technical report". In: *arXiv preprint arXiv:2303.08774*.

Angles, R. and C. Gutierrez (2008). "Survey of graph database models". In: *ACM computing surveys (CSUR)* 40.1, pp. 1–39.

Anthropic (2024). *The Claude 3 Model Family: Opus, Sonnet, Haiku*. URL: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf.

Bahari, S. F. (2010). "Qualitative versus quantitative research strategies: contrasting epistemological and ontological assumptions". In: *Sains Humanika* 52.1.

Bender, E. M., T. Gebru, A. McMillan-Major, and S. Shmitchell (2021). "On the dangers of stochastic parrots: Can language models be too big?" In: *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pp. 610–623.

Besta, M., N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al. (2024). "Graph of thoughts: Solving elaborate problems with large language models". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38. 16, pp. 17682–17690.

Bryman, A. (2003). *Research methods and organization studies*. Routledge.

Chen, X., S. Jia, and Y. Xiang (2020). "A review: Knowledge reasoning over knowledge graph". In: *Expert systems with applications* 141, p. 112948.

Cheng, F., H. Li, F. Liu, R. Van Rooij, K. Zhang, and Z. Lin (2025). "Empowering LLMs with logical reasoning: a comprehensive survey". In: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pp. 10400–10408.

Cordella, L. P., P. Foggia, C. Sansone, and M. Vento (2004). "A (sub) graph isomorphism algorithm for matching large graphs". In: *IEEE transactions on pattern analysis and machine intelligence* 26.10, pp. 1367–1372.

Creswell, J. W., V. L. Plano Clark, M. L. Gutmann, and W. E. Hanson (2003). "Advanced mixed methods research designs". In: *Handbook of mixed methods in social and behavioral research* 209.240, pp. 209–240.

Di, Z., C. Zhang, H. Lv, L. Cui, and L. Liu (2025). "Lorp: LLM-based logical reasoning via prolog". In: *Knowledge-Based Systems*, p. 114140.

Garcez, A. d. and L. C. Lamb (2023). "Neurosymbolic ai: The 3 rd wave". In: *Artificial Intelligence Review* 56.11, pp. 12387–12406.

Hogan, A., E. Blomqvist, M. Cochez, C. d'Amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, et al. (2021). "Knowledge graphs". In: *ACM Computing Surveys (Csur)* 54.4, pp. 1–37.

Ji, S., S. Pan, E. Cambria, P. Marttinen, and P. S. Yu (2021). "A survey on knowledge graphs: Representation, acquisition, and applications". In: *IEEE transactions on neural networks and learning systems* 33.2, pp. 494–514.

Kamoi, R., S. S. S. Das, R. Lou, J. J. Ahn, Y. Zhao, X. Lu, N. Zhang, Y. Zhang, H. R. Zhang, S. R. Vummanthala, et al. (2024). "Evaluating LLMs at Detecting Errors in LLM Responses". In: *First Conference on Language Modeling*.

Kapetanakis, S., G. Samakovitis, P. Gunasekera, and M. Petridis (2012). "Monitoring financial transaction fraud with the use of case-based reasoning". In: *Proceedings of the 17th UK Case-Based Reasoning Workshop*. Cambridge, UK.

Kirichenko, P., M. Ibrahim, K. Chaudhuri, and S. J. Bell (2025). "AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions". In: *ICML 2025 Workshop on Reliable and Responsible Foundation Models*.

Leon, M. (2025). "GPT-5 and open-weight large language models: Advances in reasoning, transparency, and control". In: *Information Systems*, p. 102620.

Ling, Z., Y. Fang, X. Li, Z. Huang, M. Lee, R. Memisevic, and H. Su (2023). "Deductive verification of chain-of-thought reasoning". In: *Advances in Neural Information Processing Systems* 36, pp. 36407–36433.

Luo, L., Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, and S. Pan (2025). "Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models". In: *Forty-second International Conference on Machine Learning*.

Lyu, Q., S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch (2023). "Faithful chain-of-thought reasoning". In: *Proceedings of the 13th International Joint Conference on Natural Language*

Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 305–329.

Maleki, N., B. Padmanabhan, and K. Dutta (2024). "AI hallucinations: a misnomer worth clarifying". In: *2024 IEEE conference on artificial intelligence (CAI)*. IEEE, pp. 133–138.

Morishita, T., G. Morio, A. Yamaguchi, and Y. Sogawa (2024). "Enhancing reasoning capabilities of llms via principled synthetic logic corpus". In: *Advances in Neural Information Processing Systems* 37, pp. 73572–73604.

Phan, D., M. D. Hoffman, D. Dohan, S. Douglas, T. A. Le, A. Parisi, P. Sountsov, C. Sutton, S. Vikram, and R. A Saurous (2023). "Training chain-of-thought via latent-variable inference". In: *Advances in Neural Information Processing Systems* 36, pp. 72819–72841.

Shao, Z., Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen (2023). "Synthetic prompting: Generating chain-of-thought demonstrations for large language models". In: *International conference on machine learning*. PMLR, pp. 30706–30775.

Sinha, K., S. Sodhani, J. Dong, J. Pineau, and W. L. Hamilton (2019). "CLUTRR: A diagnostic benchmark for inductive reasoning from text". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 4506–4515.

Team, G., R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, et al. (2023). "Gemini: a family of highly capable multimodal models". In: *arXiv preprint arXiv:2312.11805*.

Tharwat, A. (2021). "Classification assessment methods". In: *Applied computing and informatics* 17.1, pp. 168–192.

Tong, Y., D. Li, S. Wang, Y. Wang, F. Teng, and J. Shang (2024). "Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3065–3080.

Tyen, G., H. Mansoor, V. Cărbune, Y. P. Chen, and T. Mak (2024). "LLMs cannot find reasoning errors, but can correct them given the error location". In: *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 13894–13908.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017). "Attention is all you need". In: *Advances in neural information processing systems* 30.

Wei, J., X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. (2022). “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35, pp. 24824–24837.

Weng, Y., M. Zhu, F. Xia, B. Li, S. He, S. Liu, B. Sun, K. Liu, and J. Zhao (2023). “Large language models are better reasoners with self-verification”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 2550–2575.

Xu, F., Q. Lin, J. Han, T. Zhao, J. Liu, and E. Cambria (2025). “Are large language models really good logical reasoners? a comprehensive evaluation and beyond”. In: *IEEE Transactions on Knowledge and Data Engineering*.

Zhang, Y., Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, et al. (2025). “ Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models”. In: *Computational Linguistics* 51.4, pp. 1373–1418.